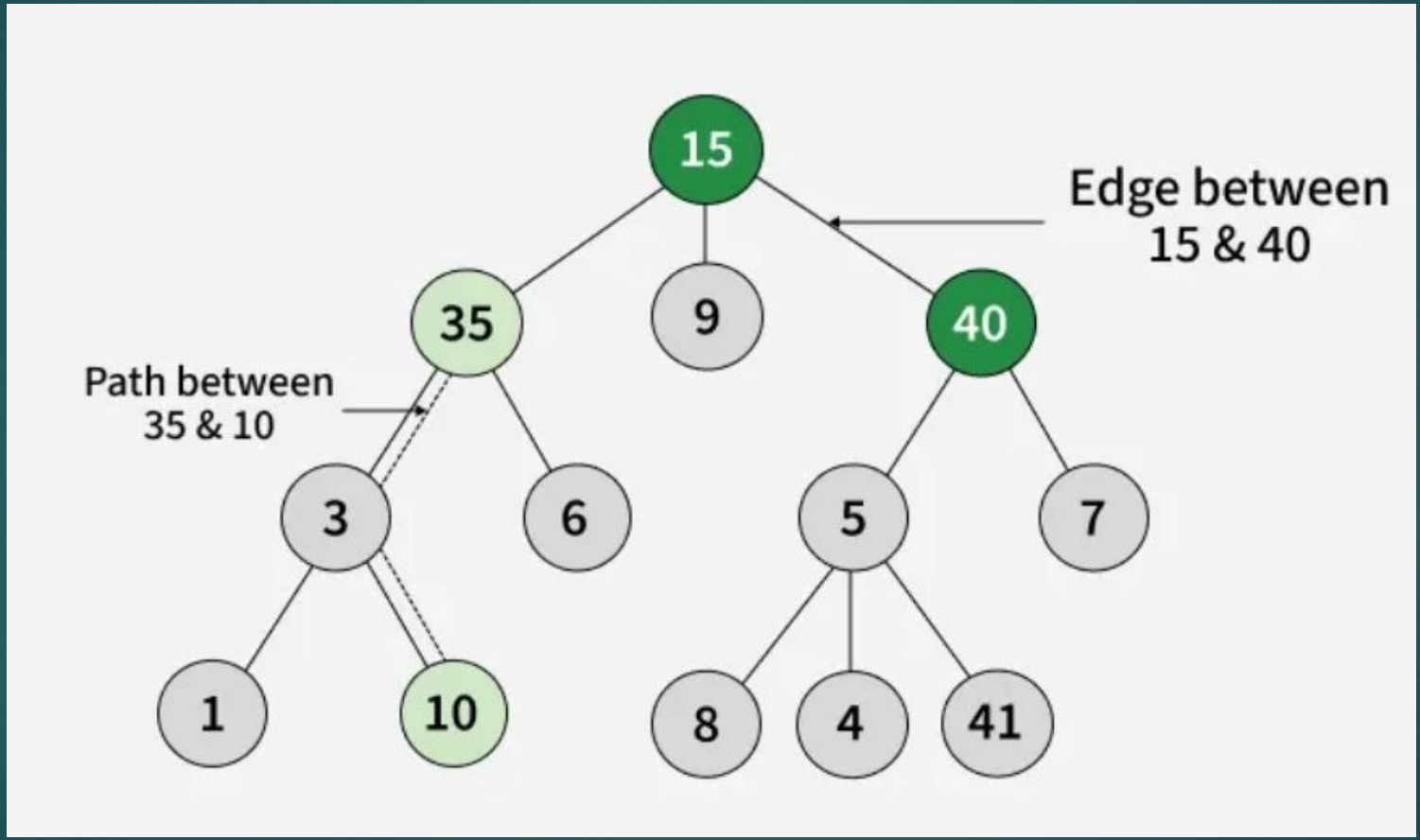
UNIT-3

NON-LINEAR DATA STRUCTURE

TREE

- A tree is a **hierarchical** data structure used to organize and represent data in a **parent–child** relationship.
- It consists of nodes, where the topmost node is called the **root**, and every other node can have one or more **child nodes**.


15
Edge between
15 & 40
35
9
40
Path between
35 & 10
3
6
5
7
1
10
8
4
41

Basic Terminologies In Tree Data Structure:

- **Parent Node:** A node that is an immediate predecessor of another node. Example: 35 is the parent of 3 and 6.
- **Child Node:** A node that is an immediate successor of another node. Example: 3 and 6 are children of 35.
- **Root Node:** The topmost node in a tree, which does not have a parent. Example: 15 is the root node.
- **Leaf Node (External Node):** Nodes that do not have any children. Example: 1, 10, 12, 5, 7, 7 are leaf nodes.
- **Ancestor:** Any node on the path from the root to a given node (excluding the node itself). Example: 15 and 35 are ancestors of 10.

.

- **Descendant:** A node x is a descendant of another node y if y is an ancestor of x. Example: 1, 10, and 6 are descendants of 35.
- **Sibling:** Nodes that share the same parent. Example: 1 and 10 are siblings, and 5 and 7 are siblings.
- **Level of a Node:** The number of edges in the path from the root to that node.The root node is at level 0.
- **Internal Node:** A node with at least one child.
- **Neighbor of a Node:** The parent or children of a node.
- **Subtree:** A node and all its descendants form a subtree

Why Tree is considered a non-linear data structure?

- Data in a tree is not stored sequentially (i.e., not in a linear order). Because of this arrangement, a tree is classified as a non-linear data structure.

Operations Of Tree Data Structure:

- **Create** – create a tree in the data structure.
- **Insert** – Inserts data in a tree.
- **Search** – Searches specific data in a tree to check whether it is present or not.

Binary trees are primarily stored and manipulated using two methods: **linked-node representation** (using pointers) and **array representation** (sequential storage).

Storage Representation

Linked-Node Representation

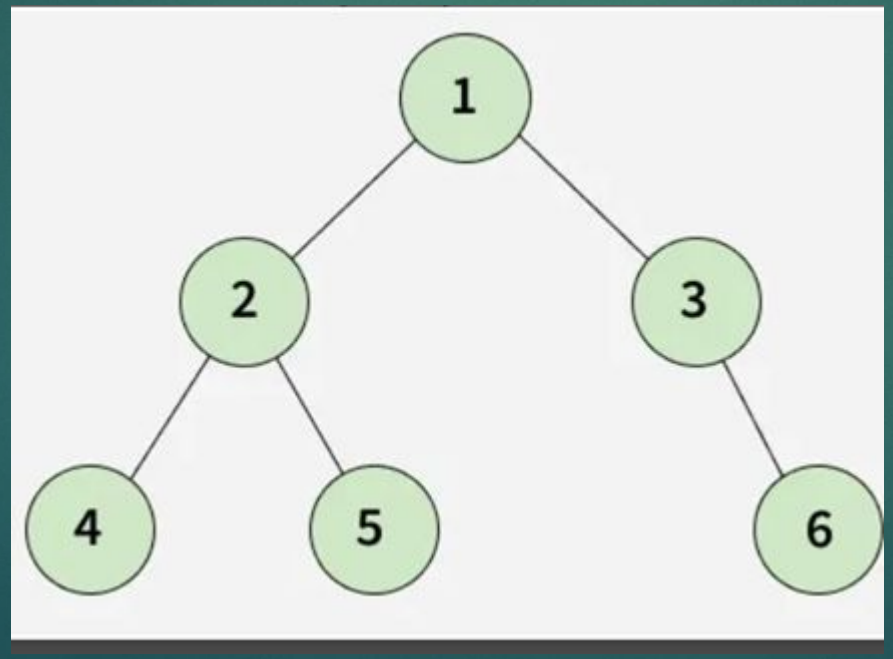
This common, flexible approach uses dynamic memory to create node objects, each containing data and pointers to left and right children

BINARY TREE

- **Insertion:** Adds new nodes, often maintaining a specific structure like a Binary Search Tree (BST) where left child < root < right child.
- **Deletion:** Removes nodes. In binary trees, this often involves replacing the node with the deepest and rightmost node to maintain structural integrity.
- **Traversals:**
  - *Inorder (Left, Root, Right):* Commonly used in BSTs to visit nodes in ascending order.
  - *Preorder (Root, Left, Right):* Used for creating copies of the tree.
  - *Postorder (Left, Right, Root):* Used for deleting the tree.
  - *Level-Order:* Visits nodes level by level (Breadth-First Search).

Preorder Traversal

The Preorder Traversal visits the nodes in the following order: Root, Left, Right. Therefore, we visit the root node 1, then the left node 2 and lastly the right node 3.



Output: *[1, 2, 4, 5, 3, 6]*

Explanation: *Preorder Traversal (Root -> Left -> Right)***.** *Visit 1 -> 2 -> 4 -> 5 -> 3 -> 6, resulting in 1 2 4 5 3 6.*

Post Order Traversal

- Given a **root** of the binary tree, return the **postorder traversal** of the binary tree.
- **Postorder Traversal** is a method to traverse a tree such that for each node, you first traverse its left subtree, then its right subtree, and finally visit the node itself.